1) Capas lógicas y niveles físicos

• Capa (layer) = división lógica. Nivel (tier) = ubicación física en una o varias máquinas.
• Presentación: interfaz e interacción con el usuario.
• Lógica de negocio: procesa solicitudes, reglas de negocio y comunicación con datos/servicios; incluye persistencia/acceso a datos.
• Datos: almacenamiento y gestión de la información.
• Cliente/servidor aporta recursos compartidos, separación funcional, reutilización, independencia, integridad, escalabilidad, interoperabilidad y transparencia.

2) Arquitecturas por niveles

Arquitectura	Distribución / clave
1 nivel	Todo en servidor/mainframe; terminales tontos; monolítica
2 niveles	Cliente + servidor; front-end/back-end
3 niveles	Cliente + servidor web/aplicaciones + SGBD
Multinivel	Servidor web + varios servidores de aplicación + datos/documentos

La arquitectura de 3 niveles es la indicada en el tema como ideal para aplicaciones web.

3) Arquitectura de 2 niveles

• Cliente: consume servicios. Servidor: procesa/provee servicios. Protocolos conectan ambos.
• Configuraciones: presentación distribuida; presentación remota = cliente ligero; lógica distribuida; datos remotos = cliente pesado; datos distribuidos.
• Ventajas: aprovecha cómputo del cliente, mejora escalabilidad frente a mononivel, presentación portable.
• Inconvenientes: sesiones/protocolos dificultan escalado; mal encaje con varios servidores; cambios pueden exigir reinstalar clientes.

4) Arquitectura de 3 niveles y multinivel

• 3 niveles: nivel 1 cliente/presentación; nivel 2 servidor web o de aplicaciones/lógica; nivel 3 SGBD/datos.
• Ventajas: reutilización, mantenimiento más sencillo, bajo acoplamiento, mayor escalabilidad y rendimiento.
• Multinivel: separa más la lógica de negocio: negocio, seguridad, BI, infraestructura, servicios y persistencia.
• Elementos típicos: navegador/cliente, frontales web balanceados, uno o varios servidores de aplicaciones y servidores de datos/documentos.
• Ventajas: escalado, compartición/reutilización, alta cohesión y mantenimiento. Inconvenientes: más tráfico, complejidad y coste.

5) SOA - Arquitectura Orientada a Servicios

• SOA organiza capacidades distribuidas como servicios reutilizables; su objetivo es integrar aplicaciones construyendo servicios, no aplicaciones finales.
• 8 principios: interoperabilidad/contrato, bajo acoplamiento, abstracción, reutilización, autonomía, ausencia de estado, descubrimiento y composición.
• Componentes: servicio, proveedor, consumidor y registro. Cada servicio posee implementación, contrato e interfaz.
• Orquestación: una entidad controla el flujo. Coreografía: colaboración pública sin controlador único.
• SOA define QUÉ; los servicios web definen CÓMO.

6) Microservicios

• Aplicación = conjunto de servicios pequeños, independientes y especializados, cada uno en su proceso y desplegable de forma independiente.
• Características: mantenibles, comprobables, débilmente acoplados, autónomos, reutilizables y organizados por capacidades de negocio.
• Ventajas: agilidad, escalado independiente, integración/entrega continua y libertad tecnológica.
• Frente a SOA: SOA es enfoque arquitectónico para toda una organización; microservicios son estrategia de implementación para una aplicación.

7) Comunicación y API Gateway

• Síncrona: petición/respuesta mediante REST/HTTP, SOAP, GraphQL o gRPC.
• Asíncrona: mensajes/eventos y colas. One-to-one = productor-consumidor; publish/subscribe = varios receptores.
• AMQP: protocolo de mensajería; bróker distribuye mensajes. Ej.: RabbitMQ y Kafka.
• API Gateway: punto único de entrada/proxy inverso; enruta, autentica, valida, transforma respuestas, gestiona tráfico y métricas. Varios gateways por cliente = patrón BFF.

8) Servicio web: concepto y tipos

• Un Web Service expone funcionalidad de un sistema mediante tecnologías estándar web y permite interacción máquina-a-máquina interoperable.
• Tipos principales del tema: SOAP (mensajes XML + contrato WSDL) y RESTful (HTTP/URI, infraestructura ligera, sin necesidad de WSDL).

9) Arquitectura SOAP

• Actores: servicio, proveedor, registro y solicitante. El proveedor publica, el cliente descubre, asocia (binding) e invoca.
• Un WS puede ser descrito (WSDL), publicado/descubierto (UDDI), asociado, invocado (SOAP) y transportado (HTTP, SMTP...).
• SOAP encaja bien cuando se requiere contrato formal, transacciones, estado, seguridad, coordinación o mensajería fiable.

10) Tecnologías SOAP

Tecnología	Función
XML	Estructura/intercambio de información
HTTP	Transporte más habitual
SOAP	Formato/protocolo de mensajes e invocación
WSDL	Descripción del servicio/interfaz
UDDI	Registro y descubrimiento
WSIL	Descubrimiento alternativo/complementario

UDDI: páginas blancas = proveedor; amarillas = categorías; verdes = información técnica.

11) SOAP: estructura del mensaje

• SOAP 1.2 es un protocolo ligero basado en XML para intercambio de información estructurada.
• Características: extensible, neutral respecto al transporte e independiente del lenguaje.
• Mensaje XML: Envelope obligatorio; Header opcional; Body obligatorio; Fault dentro del Body para errores.
• SOAP puede viajar sobre HTTP/SMTP y otros transportes; no está ligado exclusivamente a HTTP.

12) WSDL y descubrimiento

• WSDL 2.0: lenguaje XML que describe cómo llamar al servicio, parámetros y estructuras de datos devueltas.
• Separa una sección abstracta (operaciones/tipos de entrada-salida) de una sección concreta (protocolo, formato y ubicación).
• UDDI publica metainformación y URL del WSDL. WSIL permite preguntar directamente al proveedor por sus servicios.
• Operativa típica: cliente lee WSDL -> conoce operaciones/tipos -> invoca mediante SOAP.

13) REST: concepto

• REST es un estilo arquitectónico de Roy Fielding (2000), no una tecnología concreta.
• Los servicios son recursos identificados por URI; el servidor devuelve una representación del recurso (JSON, XML, HTML...).
• Cada petición contiene la información necesaria: arquitectura sin estado. Los sistemas que cumplen sus restricciones se denominan RESTful.
• REST se apoya en estándares web: HTTP, URI y representaciones; no usa SOAP ni RPC como base.

14) Restricciones REST

• Cliente-servidor; sin estado; caché; interfaz uniforme; sistema por capas; código bajo demanda (opcional).
• Interfaz uniforme: identificación de recursos, manipulación mediante representaciones, mensajes autodescriptivos y HATEOAS.
• Sin estado mejora transparencia, fiabilidad y escalabilidad; la caché reduce tráfico y tiempo de respuesta.

15) Métodos HTTP y CRUD

Método	Uso típico
GET	Recuperar recurso; seguro e idempotente
POST	Crear/procesar recurso
PUT	Establecer/reemplazar estado del recurso
DELETE	Eliminar recurso

REST centra el diseño en nombres/recursos y URIs; RPC lo centra en verbos/acciones remotas.

16) OAuth 2.0

• OAuth 2.0 es autorización, NO autenticación.
• Permite que una aplicación acceda a recursos de otra en nombre del usuario sin compartir sus credenciales.
• Usa tokens de acceso; OAuth no fija el formato del token, aunque puede usarse JWT.
• Flujo general: cliente solicita autorización -> servidor valida cliente/ámbitos -> propietario concede -> servidor devuelve código de autorización o token.

17) JWT

• JWT (RFC 7519): formato compacto y autónomo para transmitir información JSON firmada; en el tema se usa como mecanismo de autenticación sin estado.
• Se envía normalmente como Authorization: Bearer <token>.
• Estructura: HEADER.PAYLOAD.SIGNATURE, codificados en Base64Url.
• Header = tipo/algoritmo; Payload = claims; Signature = verifica integridad y, según firma, identidad del emisor.

18) OpenAPI

• OpenAPI 3 (antes Swagger): especificación independiente del lenguaje para describir APIs HTTP.
• Se escribe normalmente en YAML o JSON. Facilita comprender la API, generar clientes, configurar infraestructura y crear pruebas.
• Swagger UI genera documentación visual desde OpenAPI.
• Alternativas citadas: RAML, API Blueprint y WADL.

19) Códigos HTTP que conviene memorizar

Grupo	Ejemplos clave
1xx	100 Continue
2xx	200 OK · 201 Created · 204 No Content
3xx	301 Moved Permanently · 304 Not Modified
4xx	400 Bad Request · 401 Unauthorized · 403 Forbidden · 404 Not Found · 409 Conflict · 429 Too Many Requests
5xx	500 Internal Server Error · 502 Bad Gateway · 503 Service Unavailable · 504 Gateway Timeout

20) RESTful vs SOAP

RESTful	SOAP
Ligero; estilo arquitectónico	Más pesado; estándar/protocolo
HTTP/URI; suele usar JSON/XML	Mensajes XML + SOAP
Ideal para servicios sin estado, caché, móviles y microservicios	Ideal con contrato WSDL, estado, mensajería fiable, transacciones y seguridad avanzada
Documentación API necesaria	Herramientas permiten integración más automática

21) Frameworks / herramientas

• SOAP + REST: Apache Axis2, Apache CXF, WCF, Zend, Laravel.
• REST: Jersey, RESTEasy. SOAP: gSOAP, CodeIgniter.
• Pruebas: SoapUI (SOAP/REST) y Postman (REST).
• OpenAPI: especificación para APIs/servicios RESTful.

22) Arquitectura hexagonal

• También llamada puertos y adaptadores; busca desacoplar capas y permitir que evolucionen por separado.
• Capas: Dominio (reglas de negocio), Aplicación (casos de uso/puntos de entrada) e Infraestructura (BD, WS, UI...).
• Puerto = interfaz/contrato público. Adaptador = implementación concreta para una tecnología/contexto.
• El dominio no debe acoplarse a lo externo; aplica inversión de dependencias: depender de contratos, no de implementaciones.